

Technical Report

Design and Verification of 8-bit PISO and SIPO Shift Registers

Reece M. Dobro

Abstract

The process of designing and testing an 8-bit Parallel-Input-Serial-Output (PISO) register and an 8-bit Serial-Input-Parallel-Output (SIPO) register using the Verilog hardware description language within AMD Vivado is presented in this report. Eight simulations were created to evaluate and examine both of the modules, with a variation in inputs and time between inputs to test the modules under various conditions. The results of the simulations were analyzed using waveform outputs and TCL console logs, which confirmed that the outputs of the module properly converted the inputs into their respective formats. In addition, the RTL Schematic from AMD Vivado was examined and compared to the digital design of the modules. Overall, the simulation results demonstrate that the design operates correctly and satisfies the functional requirements specified by the project specifications.

1 Introduction, Background, and Theory

The main objective of this project is to design, program, and test an 8-bit Parallel-Input-Serial-Output (PISO) register and an 8-bit Serial-Input-Parallel-Output (SIPO) register. The functions performed by these registers are critical for collecting and sending data through communication protocols. The PISO register loads multiple bits of parallel data at once and serializes them for output, while the SIPO register receives eight 1-bit values sequentially, through a serial input, and reconstructs them into an 8-bit parallel output after multiple clock cycles. Both structures are based on shift registers and include additional logic to support other functions given by the project specifications, such as shifting operations and synchronous reset functionality.

1.1 Inputs and Outputs

The inputs, outputs, and expected behavior were provided by the project specifications for this assignment. As the modules perform different functions, they have different inputs and outputs. These inputs and outputs are shown in Tables 1 and 2.

Signal	Direction	Size	Description
clk	Input	1	System clock for synchronous data shifting
rst	Input	1	Active-high synchronous reset; clears the registers
load	Input	1	Load enable; when high, parallel data is loaded into the register
shift	Input	1	Shift enable; when high, data is shifted out serially
parallel_in	Input	8	8-bit parallel input data to be serialized
serial_out	Output	1	Serial output; outputs LSB first from the loaded parallel data

Table 1: PISO Module Port Signal and Function Table

Signal	Direction	Size	Description
clk	Input	1	System clock for synchronous data shifting
rst	Input	1	Active-high synchronous reset; clears the registers
shift	Input	1	Shift enable; when high, serial input is shifted into the register
serial_in	Input	1	Serial input data to be parallelized
parallel_out	Output	8	8-bit parallel output; updated as serial data is shifted in

Table 2: SIPO Module Port Signal and Function Table

As seen in the tables, the PISO and SIPO modules have the same number of outputs, 1, with their size differing due to PISO outputting serial data and SIPO outputting parallel data. A critical detail of the 8-bit output for the SIPO module, *Parallel_out*, is that its least significant bit (LSB) is *Parallel_out[0]*. Therefore, to ensure that the 8-bit value formed from the 1-bit serial input corresponds to the expected output, the least significant bit (LSB) of the input must be entered in first and consequently, the most significant bit (MSB) entered in last. Regarding the input configuration, PISO has an additional input, *load*, which when driven high, will assign the values of *parallel_in* to their appropriate registers. This is not needed for SIPO as the similar behavior occurs when the input, *shift*, is driven low.

1.2 Design of PISO

Figure 1 shows the "Pen and Paper" design of PISO. The design includes the names for all inputs and outputs as specified by the project specifications.

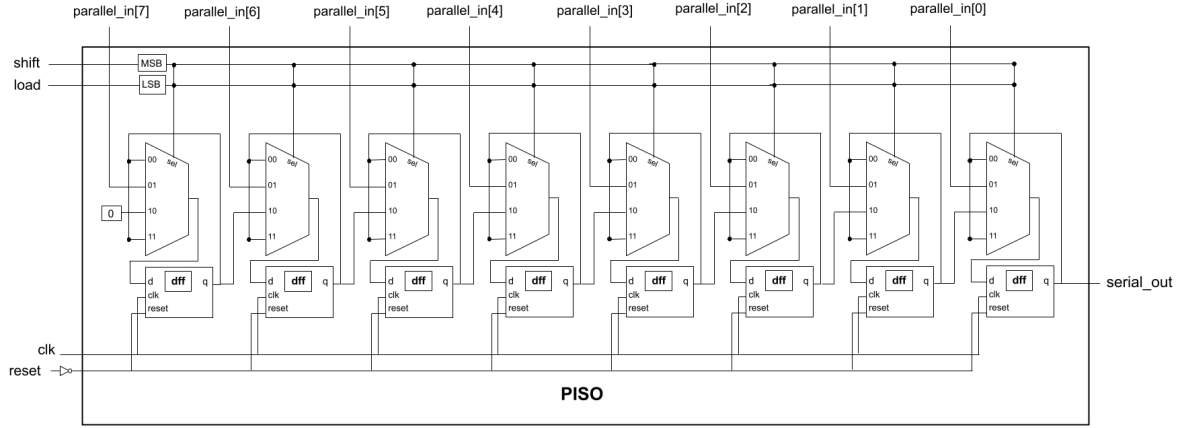


Figure 1: "Pen and Paper" Design of PISO

As shown in Figure 1, the design primarily consists of an 8-bit shift register and eight 4-to-1 multiplexers. A 4 to 1 multiplexer was chosen as there are 3 different options for the input to each D Flip-Flop. These options are controlled by the 1-bit input *shift* and the 1-bit input *load*. When *load* = 0 and *shift* = 1, the input to the D Flip Flop is the output of the previous D Flip Flop. The output of the last D Flip-Flop drives the output *serial_out*, as that value is "shifted out" from the parallel input. As the project specifications require empty locations to have a value of 0, the first D Flip-Flop will shift in 0 when *load* = 0 and *shift* = 1. This causes subsequent shifts to fill empty locations with 0. The second option that can occur for the multiplexer is when *load* = 1 and *shift* = 0. In this scenario, each bit of the 8-bit input, *parallel_in*, drives the input to the appropriate D Flip Flop. The final option that can occur is when inputs *load* and *shift* have the same value. As per project specifications, the value in each register remains the same. This behavior is performed by having the output of each D Flip-Flop drive its own input.

1.3 Design of SIPO

Figure 2 shows the "Pen and Paper" design of SIPO. The design includes the names for all inputs and outputs as specified by the project specifications.

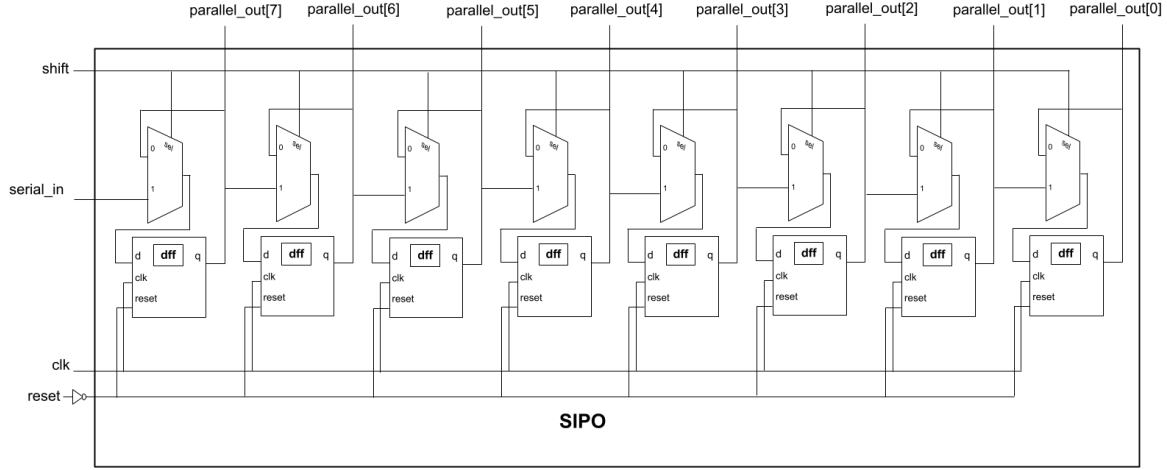


Figure 2: "Pen and Paper" Design of SIPO

As shown in Figure 2, the design primarily consists of an 8-bit shift register and eight 2 to 1 multiplexers. A 2 to 1 multiplexer was chosen as there are 2 options for the input to each D Flip-Flop. These options are controlled by the 1-bit input, *shift*. When *shift* is high, the input to a D Flip Flop is the output of the previous D Flip Flop. When *shift* is low, the input to a D Flip Flop remains the same. This behavior is performed by having the output of each D Flip-Flop drive its own input.

Both Figure 1 and Figure 2 have a NOT gate after the 1-bit input, *reset*. The specifications require an active high synchronous reset for both modules. The D flip-flop model used for this program has an active low synchronous reset, so the NOT gate was added in order to meet project specifications.

1.4 Verilog Implementation

The implementation of the PISO module and the SIPO module was achieved using both behavioral-level modeling and data flow modeling. Figure 3 shows the source code of the 4 to 1 multiplexer utilized in the PISO module. The module is written at the behavioral level; however, as it performs combinational logic, it could have been written at the dataflow level. The same design is used for the 2 to 1 multiplexer for the SIPO module, except that *sel* is a 1-bit input instead of a 2-bit input bus. In addition, *in2* and *in3* are not used for the 2 to 1 multiplexer, as only 2 inputs can be selected from. Both modules include a default case to avoid unintended latches.

```

module mux4to1(
    input [1:0] sel,
    input in0,
    input in1,
    input in2,
    input in3,
    output reg out
);
    always @(*) begin
        case (sel)
            2'b00: out = in0;
            2'b01: out = in1;
            2'b10: out = in2;
            2'b11: out = in3;
            default: out = in0;
        endcase
    end
endmodule

```

Figure 3: Source Code of Module *mux4to1*

Figure 4 shows the source code for the D Flip Flop module used for the PISO module and SIPO

module. As a D Flip Flop preforms sequential logic, the source code was written at the behavioral-level. This module also has a 1-bit input, *rst*, which is active high and synchronous.

```

module dff(
    input clk,
    input rst,
    input d,
    output reg q
);
    always@ (posedge clk) begin
        if(!rst) q<=0;
        else q<=d;
    end
endmodule

```

Figure 4: Source Code of Module *dff*

Figure 5 shows the source code for the top-level module of the SIPO program, *SIPO_8bit*. This module is based directly on the submodules shown previously and the digital design diagram, Figure 2. The module is written at the behavioral level; however, as it performs combinational logic, it could have been written at the dataflow level. The length of the program could have been reduced by removing the statements that assign the values of *q* to *parallel_out*, however, these lines improve the clarity and comprehension of the program's structure. As each bit of the parallel output requires a D flip-flop instance and a 2 to 1 multiplexer instance, 16 instances of modules are used.

```

module SIPO_8bit(
    input shift,
    input serial_in,
    input clk,
    input reset,
    output [7:0] parallel_out
);
    wire rst=!reset;
    wire [7:0] d,q;

    dff u1(clk,rst,d[0],q[0]);
    mux2tol u2(shift,q[0],serial_in,d[0]);
    dff u3(clk,rst,d[1],q[1]);
    mux2tol u4(shift,q[1],q[0],d[1]);
    dff u5(clk,rst,d[2],q[2]);
    mux2tol u6(shift,q[2],q[1],d[2]);
    dff u7(clk,rst,d[3],q[3]);
    mux2tol u8(shift,q[3],q[2],d[3]);
    dff u9(clk,rst,d[4],q[4]);
    mux2tol u10(shift,q[4],q[3],d[4]);
    dff u11(clk,rst,d[5],q[5]);
    mux2tol u12(shift,q[5],q[4],d[5]);
    dff u13(clk,rst,d[6],q[6]);
    mux2tol u14(shift,q[6],q[5],d[6]);
    dff u15(clk,rst,d[7],q[7]);
    mux2tol u16(shift,q[7],q[6],d[7]);

    assign parallel_out[7]=q[0];
    assign parallel_out[6]=q[1];
    assign parallel_out[5]=q[2];
    assign parallel_out[4]=q[3];
    assign parallel_out[3]=q[4];
    assign parallel_out[2]=q[5];
    assign parallel_out[1]=q[6];
    assign parallel_out[0]=q[7];
endmodule

```

Figure 5: Source Code of Module *SIPO_8bit*

Figure 6 shows the source code for the top-level module of the PISO program, *PISO_8bit*, which is similar to the source code of the top-level module of the SIPO program. These modules have a similar source code as the behaviors of the modules are similar, as shown by their digital designs (Figure 2 and Figure 1) being similar. The module is written at the behavioral level; however, as it performs combinational logic, it could have been written at the dataflow level. For simplicity, the 2-bit selector for the 4 to 1 multiplexers of the design is concatenated to have the 1-bit input *shift* be the most significant bit (MSB), and the 1-bit input *load* be the least significant bit (LSB). This aspect is also shown in Figure 1 by the MSB and LSB labels for the inputs to the PISO module.

```

module PISO_8bit(
    input load,
    input shift,
    input [7:0] parallel_in,
    input clk,
    input reset,
    output serial_out
);
    wire rst=!reset;
    wire [1:0] sel = {shift,load};
    wire [7:0] d,q;
    dff u1(clk,rst,d[0],q[0]);
    mux4to1 u2(sel,q[0],parallel_in[7],1'b0,q[0],d[0]);
    dff u3(clk,rst,d[1],q[1]);
    mux4to1 u4(sel,q[1],parallel_in[6],q[0],q[1],d[1]);
    dff u5(clk,rst,d[2],q[2]);
    mux4to1 u6(sel,q[1],parallel_in[5],q[1],q[2],d[2]);
    dff u7(clk,rst,d[3],q[3]);
    mux4to1 u8(sel,q[1],parallel_in[4],q[2],q[3],d[3]);
    dff u9(clk,rst,d[4],q[4]);
    mux4to1 u10(sel,q[5],parallel_in[3],q[3],q[4],d[4]);
    dff u11(clk,rst,d[5],q[5]);
    mux4to1 u12(sel,q[1],parallel_in[2],q[4],q[5],d[5]);
    dff u13(clk,rst,d[6],q[6]);
    mux4to1 u14(sel,q[6],parallel_in[1],q[5],q[6],d[6]);
    dff u15(clk,rst,d[7],serial_out);
    mux4to1 u16(sel,serial_out,parallel_in[0],q[6],serial_out,d[7]);
endmodule

```

Figure 6: Source Code of Module *PISO_8bit*

2 Simulation, Results and Analysis

To verify the accuracy of the *PISO_8bit* module and the *SIPO_8bit* module, eight distinct simulation scenarios will be executed. Each simulation will consist of two 8-bit values being entered as inputs, with the outputs of the PISO or SIPO module and their behavior being analyzed to determine if PISO or SIPO is successful. The differences between the 8 scenarios are shown in Table 3.

Simulation #	SIPO	PISO	Shift	Delay
1	✓			
2	✓		✓	
3	✓			✓
4	✓		✓	✓
5		✓		
6		✓	✓	
7		✓		✓
8		✓	✓	✓

Table 3: Simulation Summary Table for SIPO and PISO Modules

As shown in the table, the shift aspect that both modules have will be tested in half of the simulations to test the shift functionality. The delay column signifies that a delay will be added between entering the first 8-bit value and the second 8-bit value. This is done to ensure the stability of the timing logic for both the modules and that undriven serial/parallel inputs will not cause the design to fault.

2.1 Simulation Test Inputs

The first 8-bit value entered into the simulations will be 8'b1010 1010. This alternating set of bits was chosen for testing because it tests several critical behaviors for both the *PISO_8bit* module and the *SIPO_8bit* module. By having a different MSB and LSB, the value ensures that the output occurs in the correct sequence, with the PISO module outputting the LSB first and the SIPO module having the output of its LSB match the input's LSB. By having the bits of the input alternate, it ensures that the D Flip-Flops of both modules can handle continuous updates.

The second 8-bit value entered into the simulations is 8'b 1100 1110. This set of bits was chosen for testing as it complements the first 8-bit value by having its LSB, 1'b0, be different from the MSB

of the first value, 1'b1. This contrast allows for clear observation of changes in the serial output of the *PISO_8bit* module, verifying if a delay occurs when it should. This value also produces noticeable changes in the output when shifted left or right, allowing the behavior of the shift operation for the PISO and SIPO modules to be easily observed and verified.

2.2 Simulation Testbenches

All eight of the simulations for testing the modules will be conducted using a standard Verilog testbench. For all of these testbenches, the following conditions will be maintained:

- The time unit of each simulation be 1 ns.
- The period of the 1-bit input *clock* will be 10 ns, starting at 0.
- Before the start of a simulation, *reset* will be set high for 10 ns then remain low for the rest of the simulation.
- If a SIPO simulation has a delay then the time between the last bit of the first input and the first bit of the second input will be at least 50 ns.
- If a PISO simulation has a delay then the time between the first and second input will be at least 50 ns.
- If the input is serial, the delay between inputs will be 10 ns.

These conditions will be maintained to ensure uniformity for each simulation and were made in accordance with project specifications. These conditions also allow for consistent timing among simulations and prevent propagation errors and race conditions. The timing values for these conditions were chosen to ensure that all inputs were given to the module on the positive edge of the clock so it could be correctly processed.

```

module SIPO_8bit_tb;
reg shift;
wire [7:0] parallel_out;
reg clk;
reg reset;
reg serial_in;
SIPO_8bit dut(shift, serial_in, clk, reset, parallel_out);
initial begin
    clk = 0; forever #5 clk = ~clk; end
initial begin
    reset = 1;
    #10 reset = 0; shift = 1; serial_in = 0; $write("First value: %b", serial_in);
    #10 serial_in = 1; $write("%b", serial_in);
    #10 serial_in = 0; $write("%b", serial_in);
    #10 serial_in = 1; $write("%b", serial_in);
    #10 serial_in = 0; $write("%b", serial_in);
    #10 serial_in = 1; $write("%b", serial_in);
    #10 serial_in = 0; $write("%b", serial_in);
    #10 serial_in = 1; $write("%b\n", serial_in);
    #7 shift=0;
    #3 $display("Parallel_Out = %b", parallel_out);
    #50; shift=1;

    serial_in = 0; $write("Second value: %b", serial_in);
    #10 serial_in = 1; $write("%b", serial_in);
    #10 serial_in = 1; $write("%b", serial_in);
    #10 serial_in = 1; $write("%b", serial_in);
    #10 serial_in = 0; $write("%b", serial_in);
    #10 serial_in = 0; $write("%b", serial_in);
    #10 serial_in = 1; $write("%b", serial_in);
    #10 serial_in = 1; $write("%b\n", serial_in);
    #10 shift=0;
    #20 $display("Parallel_Out = %b", parallel_out);
    $finish;
end
endmodule

```

Figure 7: Source code of the Testbench for Simulation 4.

Figure 7 shows the source code of the testbench for Simulation 4. As indicated by Table 3, this simulation tests the SIPO module and has shifting enable along with a delay between the first 8-bit value and the second 8-bit value. The first input bit for an 8-bit test values is its LSB and the last input bit for the 8-bit test values is the MSB. All testbenches use the same clock cycle shown in Figure 7, beginning at 0 with a period of 10 ns as per the conditions for testbenches set previously. `$write` is used to display the input, `serial_in` to the testbench. `$write` is used as unlike, `$display`, a new line is not added after every command. As this simulation is testing the SIPO module, the delay between the last bit of the first input and the first bit of the second input is 60 ns, with the first output for the module being displayed 10 ns after the last bit of first input is entered.

This source code follows the same structure as simulations 1-3, which also have the SIPO module under test. The only differences between the source codes are the 1-bit input, `shift` being declared as high or low, and a delay being present between the two main sets of input values. This source code shows `shift` being declared high and a delay being present.

Figure 8 shows the source code of the testbench for Simulation 8. As indicated by Table 3, this simulation tests the PISO module and has shifting enabled along with a delay between the first 8-bit value and the second 8-bit value. `$write` is used to display the output, `serial_in` to the testbench. As this simulation is testing the PISO module, the 1-bit input `hold` is high while `shift` is low at the start, when the 8-bit input is given, in order to ensure that the `hold` is given to the PISO module during the positive edge of a clock signal. In order to simulate a delay, the shift is low, and the load is low. This occurs for 53 ns in order to meet the conditions outlined previously.

```

module PISO_8bit_tb;
    reg load;
    reg shift;
    reg [7:0] parallel_in;
    reg clk;
    reg reset;
    wire serial_out;
    PISO_8bit dut(load, shift, parallel_in, clk, reset, serial_out);
    initial begin
        clk = 0; forever #5 clk = ~clk; end
    initial begin
        reset = 1;
        #10 reset = 0;
        #3 parallel_in = 8'b10101010; load = 1; shift = 0;
        #2 load = 0; shift = 1;
        #10 $write("Serial output: %b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #7 load = 0; shift = 0;
        #3 $write("%b\n", serial_out);
        #50 parallel_in = 8'b11001110; load = 1; shift = 0;
        #10 load = 0; shift = 1;
        #10 $write("Serial output: %b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b", serial_out);
        #10 $write("%b\n", serial_out);
        #10 $finish;
    end
endmodule

```

Figure 8: Source code of the Testbench for Simulation 8.

This source code follows the same structure as simulations 5-7, which also have the PISO module under test. The only differences between the source codes are the 1-bit input, *shift* being declared as high or low, and a delay being present between the two main sets of input values. This source code shows *shift* being declared high and a delay being present.

2.3 Simulation Results

Tables 4 and 5 contain the results of the eight simulations compared to the expected results. The TCL Console output from the simulations was compiled into the tables to show the actual outputs and allow for direct comparison to the expected outputs. For the PISO simulations, the TCL Console outputs the results in serial as LSB to MSB, representing the bits captured over eight consecutive clock cycles. This is due to project specifications and is the expected behavior. For Table 5, the outputs were adjusted to be MSB to LSB, as that is how binary is conventionally read. For example, simulation 8's TCL Console output for the second input set was 01110011, but was recorded in Table 5 as 11001110. As shown by the tables, the expected outputs perfectly match the actual outputs of the simulations.

Sim #	Set 1 (10101010)		Set 2 (11001110)	
	Expected Outputs	Actual Outputs	Expected Outputs	Actual Outputs
1	00000000	00000000	00000000	00000000
2	10101010	10101010	11001110	11001110
3	00000000	00000000	00000000	00000000
4	10101010	10101010	11001110	11001110

Table 4: SIPO Simulation Results for Two Input Sets

Sim #	Set 1 (10101010)		Set 2 (11001110)	
	Expected Outputs	Actual Outputs	Expected Outputs	Actual Outputs
1	00000000	00000000	00000000	00000000
2	10101010	10101010	11001110	11001110
3	00000000	00000000	00000000	00000000
4	10101010	10101010	11001110	11001110

Table 5: PISO Simulation Results for Two Input Sets

The waveforms output of the testbench for simulation 4, executing the SIPO module with delay and shift, is shown in Figure 9. As shown in the Figure, the delay between the two 8-bit values being entered occurs from 87 ns to 140 ns. Despite the clock running during this time, the output, *parallel_out[7:0]*, remains constant. This demonstrates the functionality of the 1-bit input, *shift*, and that it is routinely being checked by the SIPO module. The first input set, 8'b10101010, is processed by the SIPO module, from 15 ns to 85 ns. During this time, the figure shows a 1-bit signal, originally occurring from 25 ns to 35 ns, moving diagonally from *parallel_out[7]* to *parallel_out[0]*. This 1-bit single is the LSB+1 of the first 8-bit input, 1'b1. It is shown to transfer from the MSB of *parallel_out* to its LSB+1. This behavior confirms the correct behavior and logic for the SIPO design as it shifts a value from the input into the correct Flip-Flop. The behavior of this simulation is very similar to the behavior found for simulation 2, where no delay was performed. As shown in Table 4, the actual outputs for both simulations are the same. The main difference between the two was that simulation 4's output for set 1 was extended due to it having a delay.

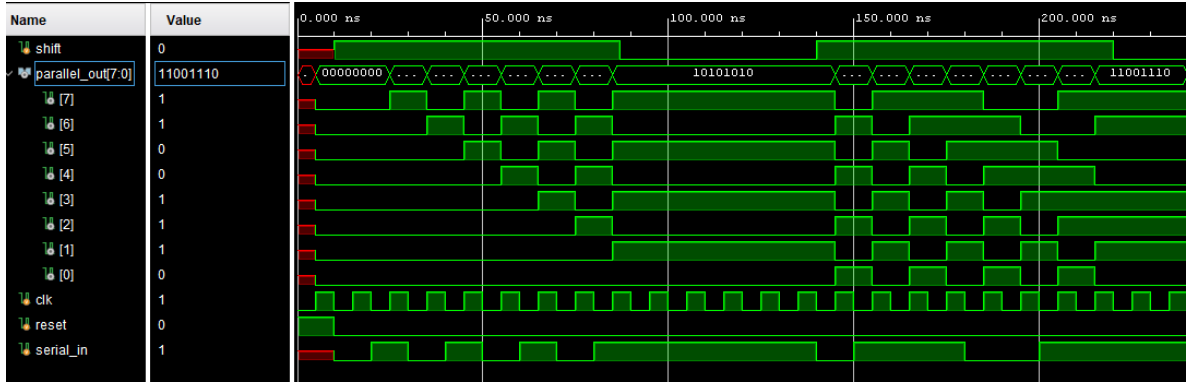


Figure 9: Waveforms Output of Simulation 4

The waveforms output of the testbench for simulation 1, executing the SIPO module with no delay and no shift, is shown in Figure 10. As shown in the Figure, the values are entered LSB first, with the first input set, 8'b10101010, being processed by the SIPO module, from 15 ns to 85 ns. The second input set is processed as soon as the next positive edge of the clock occurs, at 95 ns, as no delay occurs. After the reset is processed by the module, all bits of the output *parallel_out* remain low for the duration of the simulation. This is the correct behavior for the SIPO module, as when the module is not shifting values, it holds the current value. As *reset* sets the value of all D Flip-Flops to low, the module continuously outputs low as it never updates from the input, *serial_in*. As a result, this behavior confirms that the reset functionality and hold behavior of the SIPO design and its implementation operate as intended. The behavior of this simulation matches the behavior found for simulation 3, where no shift was performed, but a delay did occur. The only difference between the two simulations for their waveforms output was the length of the simulation itself.

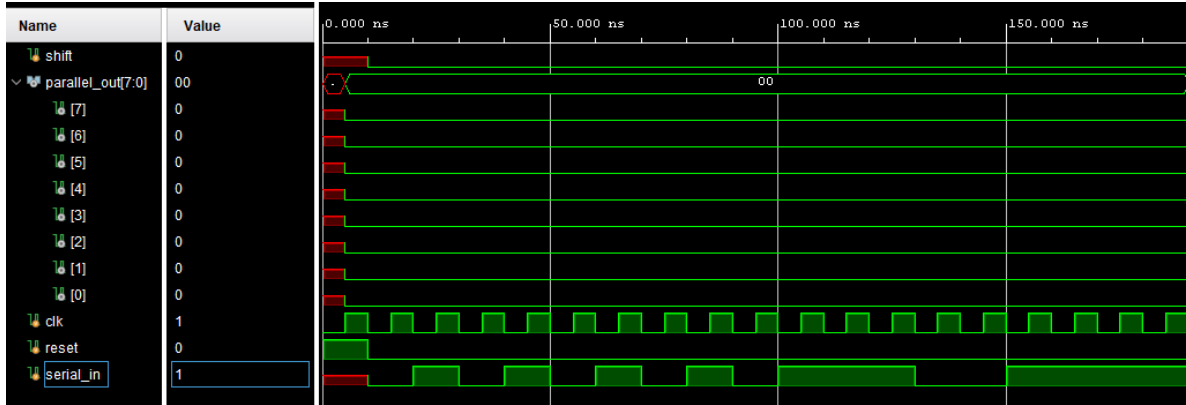


Figure 10: Waveforms Output of Simulation 1

The waveforms output of the testbench for simulation 8, executing the PISO module with delay and shift, is shown in Figure 11. As shown by the figure, from 92 ns to 145 ns, the 1-bit inputs *shift* and *load* are held low. This is done to simulate a delay. During this period, the module's serial output is 1'b1, with the module continuously "recycling" the value every positive clock edge. This behavior matches the project specifications and verifies a key requirement. In addition, at 235 ns (the last positive edge of the clock), the serial output becomes 1'b0 from 1'b1. This confirms that 1'b0 is being "shifted into" empty D Flip-Flops, as the final D Flip-Flop connected to the output goes low once eight cycles occur. The timing, eight cycles, is critical as it is the length for all input bits to be processed and output by the PISO module. The behavior of this simulation is very similar to the behavior found for simulation 6, where no delay was performed. As shown in Table 5, the actual outputs for both simulations are the same. The main difference between the two was that simulation 6's final output bit for the set 1 input was extended due to it having a delay.

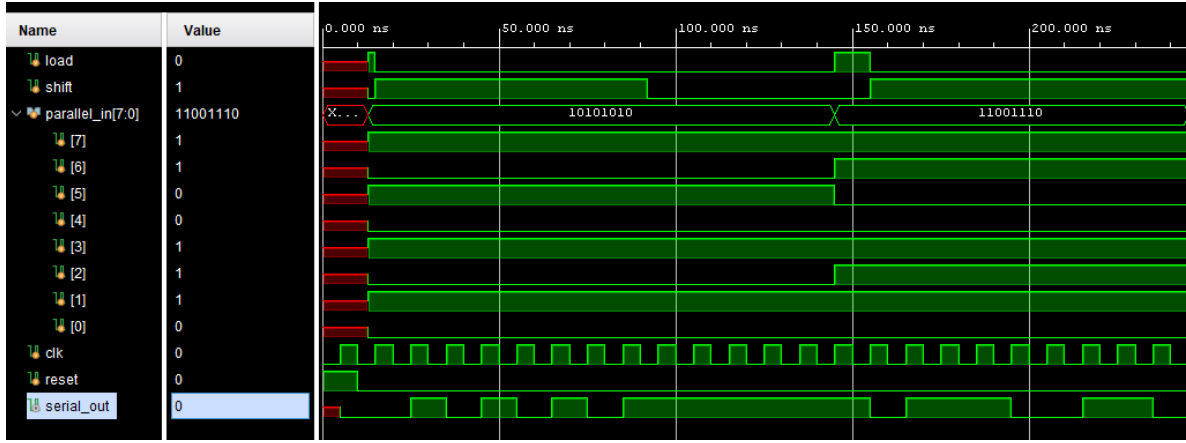


Figure 11: Waveforms Output of Simulation 8

The waveforms output of the testbench for simulation 5, executing the PISO module with no delay and no shift, is shown in Figure 12. Both sets of inputs are loaded into the module; however, the input *shift* remains low for the entirety of the simulation after reset. This is the correct behavior for the PISO module, as when the module is not shifting values, it holds the current value. As *reset* sets the value of all D Flip-Flops to low, the module continuously outputs low as it never updates from the input, *serial_in*. As a result, this behavior confirms that the reset functionality and hold behavior of the PISO design and its implementation operate as intended. The behavior of this simulation matches the behavior found for simulation 7, where no shift was performed, but a delay did occur. The only difference between the two simulations for their waveforms output was the length of the simulation itself.

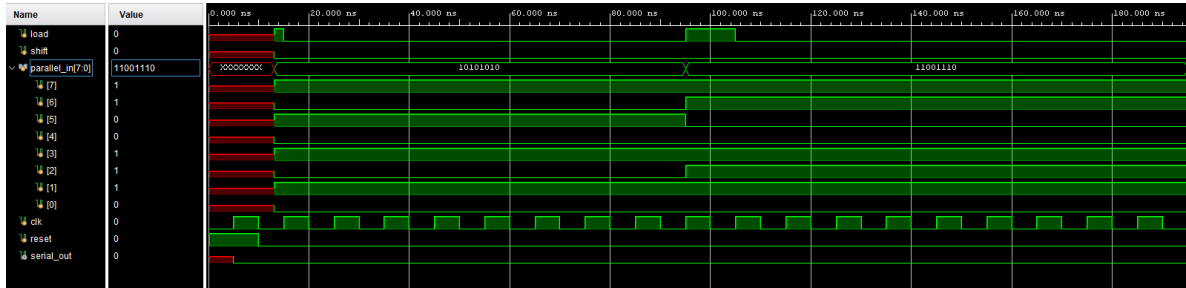


Figure 12: Waveforms Output of Simulation 5

Figures 9 to 12 have undefined values from the start of the simulation to 5 ns or 10 ns. This is shown by the red waveforms. This behavior occurred for all simulations. The reason for this behavior depends on whether the value expressed is an input or an output. For inputs, this occurs due to certain input values not being driven until 10 ns into the simulation. For the outputs, this occurs due to the active reset signal at the start of the simulation being processed by the modules when the first positive edge of the clock occurs. As the clock starts low and has a period of 10 ns, outputs are reset at 5 ns into the simulation. This initial delay is an inherent characteristic of the testbench's structure rather than a functional error. From this analysis, it can be concluded that the initial unknown state of testbench inputs and outputs is expected behavior rather than a result of a functional error in the design.

A potential limitation in the verification process is that only a limited number of test cases were utilized. For the 8-bit input to the *PISO_8bit* module, there are $2^8 = 256$ possible input combinations. Similarly, for the *SIPO_8bit* module, the serial input can form $2^8 = 256$ possible 8-bit sequences. When including the control inputs (*load*, *shift*, and *reset*), the total number of possible input combinations per clock cycle increases further. While the simulation scenarios cover the primary operations, they represent only a fraction of the possible input sequences over time. Additionally, both modules contain sequential logic, meaning their outputs depend not only on current inputs but also on previously

stored values within the register. As a result, the number of possible input-output sequences grows exponentially over multiple clock cycles. However, it is impractical to simulate every possible sequence due to the time required for manual verification and documentation. This is why representative test cases, specifically the ideal and non-ideal input sequences, selected to confirm the success of the design under a variety of conditions.

Both modules are deemed successful as the output waveforms and TCL binary logs perfectly match the expected behavior set by the project specifications and match the expected outputs.

2.4 RTL Schematic Results and Comparison

Figures 13 and 14 show the schematic of the RTL Analysis from AMD Vivado for both modules, with Figure 13 showing the SIPO module and Figure 14 showing the PISO module. These figures provide a view of the synthesized design as interpreted by the AMD Vivado. It shows the Verilog program translated into basic hardware components such as flip-flops, multiplexers, and logic gates. For Figure 13, one instance of the D Flip Flop and the 2 to 1 multiplexer is expanded to provide enhanced information about the schematic.

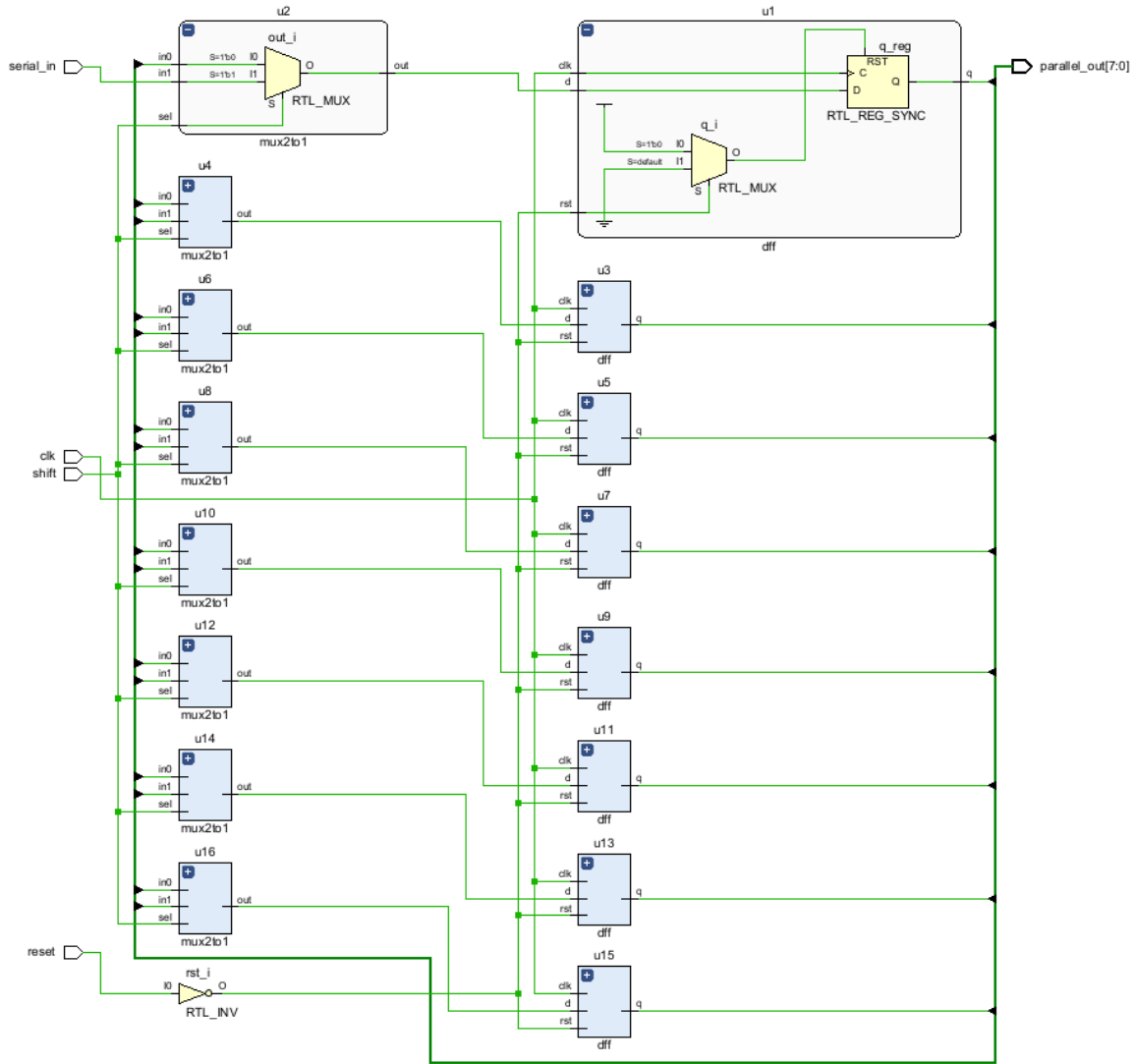


Figure 13: Synthesis result for the SIPO Module

The schematic of the SIPO Module, Figure 13, matches closely with the expected results from

the digital diagram created for the SIPO Module, Figure 2. Some of the elements shared by the two designs include eight instances of D Flip-Flops and eight instances of 2 to 1 multiplexers being used, the output of D Flip-Flops driving multiplexer inputs, and the output of each multiplexer driving the input to a D Flip-Flop instance. In addition, both designs have a NOT gate between the input port *reset* and the reset of the D-Flip Flops. The main difference between the schematic and the digital diagram is the reset logic for the D Flip-Flop instance. The digital diagram has a reset port for the D Flip-Flop instance, while the schematic shows an internal multiplexer that performs the same operation. The reason for this additional multiplexer is explained by Figure 4, the source code of the module, *dff*. As the source code uses if/else logic to control reset logic, the schematic included it in the form of a 2-to-1 multiplexer. The main difference between the two designs occurred due to the use of behavioral level modeling for the Verilog implementation. Dataflow modeling was used for the majority of the program, and its usage didn't present any differences between the two designs.

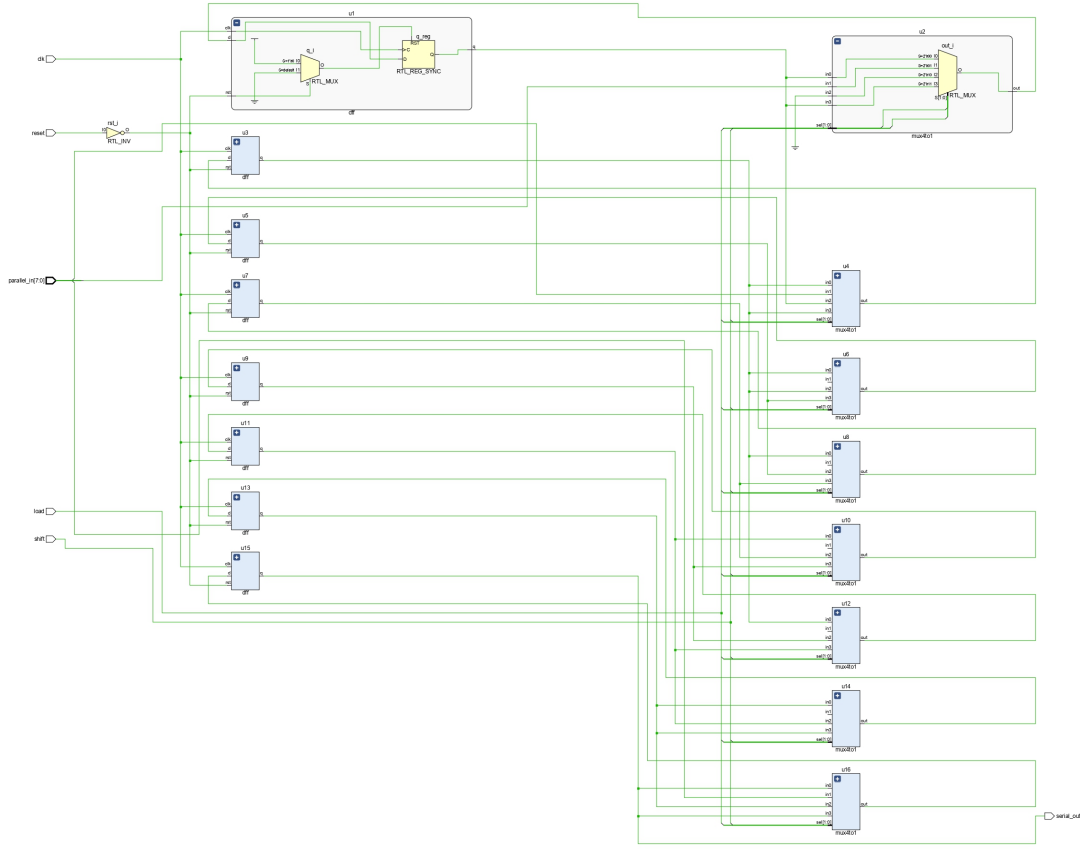


Figure 14: Synthesis result for the PISO Module

The schematic of the PISO Module, Figure 14, matches closely with the expected results from the digital diagram created for the PISO Module, Figure 1. Some of the elements shared by the two designs include eight instances of D Flip-Flops and eight instances of 4 to 1 multiplexers being used, the output of D Flip-Flops driving multiplexer inputs, and the output of each multiplexer driving the input to a D Flip-Flop instance. In addition, both designs have a NOT gate between the input port *reset* and the reset of the D-Flip Flops. The main difference between the schematic and the digital diagram is the reset logic for the D Flip-Flop instance. This is the same difference present for the SIPO Module, with both modules using the same D Flip-Flop source code.

3 Conclusions

The correct design, implementation, and verification of the *PISO_8bit* and *SIPO_8bit* modules was achieved in this project. Simulation results confirmed that the PISO module correctly turned the 8-bit parallel inputs into a single serial output, while the SIPO module accurately turned the serial inputs into the expected 8-bit parallel outputs. In addition to being functionally verified, the timing logic for both modules was shown to operate successfully while meeting all expected behaviors outlined in the project specifications. The implementation was shown to be successful, with the RTL schematic matching the digital design. Future work could include resource analysis to identify potential hardware modifications that would improve overall efficiency. Overall, this project demonstrates how Verilog can be used to implement and verify sequential hardware modules digitally, producing modular designs without requiring physical components or additional hardware.

This report was compiled using L^AT_EX .